

Rapport : K-means Adaptatif par Dispersion

1 Introduction

L'algorithme **K-means** est l'un des algorithmes de clustering les plus populaires grâce à sa simplicité et son efficacité. Cependant, il présente une limitation majeure : il nécessite de fixer à l'avance le nombre de clusters k . Dans la plupart des applications réelles (analyse de comportements utilisateurs, segmentation client, détection de profils, etc.), ce nombre est inconnu.

Un mauvais choix de k conduit à des regroupements sous-optimaux ou artificiels. De plus, l'algorithme classique est sensible à l'initialisation et ne s'adapte pas bien aux clusters de densités ou de tailles très différentes.

Objectif du projet : Concevoir et implémenter un **K-means adaptatif** capable d'ajuster automatiquement le nombre de clusters pendant l'apprentissage grâce à des mécanismes de scission (*split*), fusion (*merge*) et pondération par dispersion.

2 Principe de l'algorithme proposé

Nous avons développé **AdaptiveDispersionKMeans**, une extension du K-means classique qui intègre trois mécanismes clés :

Mécanisme	Rôle	Avantage principal
Pondération par dispersion	Les clusters denses (faible dispersion) attirent plus fortement les points lors de l'assignation	Meilleure gestion des clusters de densités variables
Scission (Split)	Divise un cluster trop dispersé en deux sous-clusters	Permet de découvrir des sous-groupes naturels
Fusion (Merge)	Regroupe les clusters trop proches	Évite la fragmentation excessive

3 Déroulement de l'algorithme

Initialisation avec un K-means classique (k_init fourni par l'utilisateur ou déduit de la variable cible).

À chaque itération :

- Calcul de la dispersion moyenne de chaque cluster (distance euclidienne moyenne au centroïde).
- Réaffectation des points avec pondération par dispersion.
- Fusion des clusters trop proches.
- Scission des clusters trop étalés (dispersion > seuil).
- Mise à jour des centroïdes.

Convergence lorsque le nombre de clusters et les labels se stabilisent.

4 Implémentation

Le projet est entièrement implémenté en Python et est **générique** (fonctionne sur différents datasets CSV).

4.1 Fonctionnalités principales

- Chargement et prétraitement automatique : suppression de colonnes inutiles, encodage one-hot des variables catégorielles, standardisation.
- Comparaison systématique entre **K-means classique** et **Adaptive Dispersion K-Means**.
- Évaluation quantitative : **Silhouette Score**, **ARI** (Adjusted Rand Index) et **NMI** (Normalized Mutual Information) lorsque une variable cible est disponible.
- Visualisation en 2D via réduction PCA.
- Historique des itérations (évolution du nombre de clusters et de la dispersion moyenne).

Le code est structuré en classes et fonctions modulaires, ce qui facilite la maintenance et les extensions futures.

4.2 Amélioration apportée : Calcul du seuil de fusion (Merge)

Lors de la revue du code, l'enseignant nous a recommandé d'améliorer le calcul du seuil de fusion dans la méthode `_merge()`.

Version initiale : Le seuil était calculé comme la **moyenne** des distances au plus proche voisin :

```
threshold = nearest[np.isfinite(nearest)].mean()
```

Ce choix était sensible aux valeurs extrêmes, ce qui pouvait entraîner une fusion trop agressive ou insuffisante.

Version améliorée : Nous avons remplacé la moyenne par la **médiane** des distances au plus proche voisin :

```
valid_nearest = nearest[np.isfinite(nearest)]
```

```
threshold = np.median(valid_nearest)
```

Justification : La médiane est une mesure de tendance centrale beaucoup plus **robuste** que la moyenne face aux valeurs aberrantes. Ce changement rend l'algorithme plus stable, particulièrement lorsque les clusters présentent des dispersions très hétérogènes. Cette modification a contribué à améliorer la qualité des regroupements finaux.

4.3 Autres améliorations techniques

4.3.1 Stratégie de scission (Split) révisée :

Dans la version initiale, tous les clusters éligibles étaient scindés simultanément, ce qui pouvait déstabiliser la convergence. Nous scindons désormais **un seul cluster** par itération : celui ayant la plus grande dispersion. Cela rend l'évolution plus progressive et contrôlée.

4.3.2 Inversion de l'ordre Merge - Split L'ordre a été modifié :

Merge d'abord, puis **Split**. Cette inversion permet de simplifier les clusters proches avant d'en créer de nouveaux, améliorant ainsi la stabilité et réduisant le nombre d'itérations nécessaires à la convergence.

4.3.3 Réindexation des labels :

Une méthode `_reindex_labels()` a été ajoutée pour garantir des indices de clusters contigus (0, 1, 2, ...) après chaque fusion ou scission, évitant ainsi des erreurs dans le calcul des centroïdes.

5 Résultats

Sur le dataset `user_behavior_dataset.csv` :

- Le **K-means classique** (avec k fixé) produit des résultats corrects mais rigides.
- Notre version **adaptative** découvre automatiquement un nombre de clusters plus pertinent.
- Améliorations observées sur :
 - Le **Silhouette Score** (meilleure cohésion et séparation).
 - La robustesse face à une mauvaise initialisation.
 - L'adaptation aux clusters de densités inégales.

La projection PCA 2D montre clairement que l'approche adaptative produit des regroupements plus naturels et mieux séparés.

6 Conclusion

Ce projet démontre qu'il est possible d'améliorer significativement le K-means classique en lui donnant la capacité de **découvrir automatiquement** la structure réelle des données.

L'introduction de la pondération par dispersion, combinée aux mécanismes dynamiques de *split* et *merge*, rend l'algorithme plus flexible et mieux adapté aux besoins réels en analyse de données non supervisée.

L'approche reste simple à comprendre et à implémenter, tout en offrant des gains notables en termes de qualité de clustering et de praticité d'utilisation.